



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش نهم

پیاده‌سازی حافظه‌ی شرکت‌پذیر سه‌گانه (TCAM)

خلاصه

در این آزمایش یک پیمانه‌ی TCAM به اندازه‌ی ۱۶ ثابت ۱۶ بیتی طراحی و پیاده‌سازی شده است. حافظه‌ی معمولی با آدرس کار می‌کند: آدرس می‌دهی و محتوا می‌گیری. حافظه‌ی شرکت‌پذیر (CAM) برعکس عمل می‌کند: محتوا می‌دهی و آدرس می‌گیری، و این کار را با مقایسه‌ی هم‌زمان همه‌ی خانه‌ها انجام می‌دهد. TCAM یک قدم جلوتر می‌رود و اجازه می‌دهد در هر خانه، علاوه بر ۰ و ۱، نماد سوم X هم ذخیره شود که یعنی «این جایگاه مقایسه نشود».

نماد سوم چطور ذخیره می‌شود

سخت‌افزار سطح منطقی سومی برای ذخیره کردن ندارد، پس هر بیت با دو فلیپ‌فلاپ نگه داشته می‌شود:

نماد ذخیره‌شده	value	care
۰	۰	۱
۱	۱	۱
X (مقدار بی‌اهمیت است)	—	۰

و یک جایگاه وقتی با بیت جست‌وجو موافق است که:

$$\text{agree} = \overline{\text{care}} + (\text{value} \oplus \text{search})$$

یعنی یا اصلاً قرار نیست مقایسه شود، یا مقایسه شده و برابر درآمده است. کل سلول سه‌گانه همین است:

```

1 // One ternary bit: two flip-flops and a three-gate comparator.
2 //
3 // This is the whole idea of a TCAM in miniature. `care` decides whether the
4 // position takes part in the comparison at all; when it is 0 the cell agrees
5 // with anything, which is the stored symbol X.
6 module tcam_cell (
7     input wire Clk,
8     input wire RstN,
9     input wire we,           // write this cell
10    input wire d,             // value to store
11    input wire care,          // 1 = compare, 0 = store X
12    input wire s,             // search bit
13    output wire agree,        // this position does not object to s
14    output wire d_q,          // stored value (observation)
15    output wire care_q        // stored care (observation)
16);
17    reg vq, cq;
18
19    always @(posedge Clk or negedge RstN) begin
20        if (!RstN) begin
21            vq <= 1'b0;
22            cq <= 1'b0;           // reset stores X, hence the validity bit above it
23        end else if (we) begin
24            vq <= d;
25            cq <= care;
26        end
27    end
28
```

```

29 assign agree = ~cq | ~(vq ^ s);
30 assign d_q   = vq;
31 assign care_q = cq;
32 endmodule

```

چرا بیت اعتبار اختیاری نیست

نکته‌ای که در نگاه اول دیده نمی‌شود ولی مدار را از کار می‌اندازد این است: خانه‌ای که همه‌ی بیت‌های care آن صفر باشد، کاملاً X است و با هر کلمه‌ای منطبق می‌شود.

پس از ریست، فلیپ‌فلاپ‌های care صفرند. بنابراین یک TCAM خالی — بدون بیت اعتبار — برای هر ورودی، شانزده انطباق گزارش می‌کند. بیت اعتبار در هر خانه دقیقاً جلوی همین را می‌گیرد و به‌عنوان یک ورودی جداگانه‌ی نوشتن هم بیرون داده شده است، تا بشود یک قاعده را بازنشسته کرد بدون اینکه محتوایش پاک شود.

$$\text{match}_i = \text{valid}_i \wedge (\bigwedge_k \text{agree}_{i,k})$$

این ویژگی در تست‌بنچ صریحاً بررسی شده است: هم اینکه TCAM خالی هرگز انطباق نمی‌دهد، و هم اینکه خانه‌ی تماماً X پس از معتبر شدن با هر کلمه‌ای منطبق می‌شود.

```

1  `include "tcam_pkg.vh"
2
3  // One 16-bit TCAM word: sixteen ternary cells plus a validity bit.
4  //
5  // The match line is a plain AND of every cell's agreement. Note that it is
6  // combinational all the way from the search word to the match line -- the
7  // storage is clocked, the lookup is not. That is what makes a CAM a CAM:
8  // every entry is compared at the same time, so a lookup costs one gate delay
9  // chain instead of sixteen memory reads.
10 module tcam_entry (
11     input wire          Clk,
12     input wire          RstN,
13
14     input wire          we,
15     input wire [15:0] wdata, // value bits
16     input wire [15:0] wmask, // 1 = compare, 0 = X
17     input wire          wvalid,
18
19     input wire [15:0] search,
20     output wire      match,
21
22     output wire [15:0] q_value, // observation ports
23     output wire [15:0] q_mask,
24     output wire      q_valid
25 );
26 wire [15:0] agree;
27 reg         valid;
28
29 genvar i;
30 generate
31     for (i = 0; i < 16; i = i + 1) begin : g_cell
32         tcam_cell u_cell (
33             .Clk(Clk), .RstN(RstN), .we(we),
34             .d(wdata[i]), .care(wmask[i]), .s(search[i]),
35             .agree(agree[i]), .d_q(q_value[i]), .care_q(q_mask[i])
36         );
37     end
38 endgenerate
39
40 always @(posedge Clk or negedge RstN) begin
41     if (!RstN) valid <= 1'b0;
42     else if (we) valid <= wvalid;
43 end
44
45 assign match = valid & (&agree);
46 assign q_valid = valid;
47 endmodule

```

مثال صورت آزمایش

صورت آزمایش می‌گوید اگر داده‌ی 01101110 در TCAM جست‌وجو شود، هر سه الگوی زیر انطباق دارند. رمزگذاری مقدار و ماسک هر کدام به این شکل درمی‌آید:

الگو	care	value	نتیجه برای 01101110
X1101XXX	01111000	01101000	انطباق
0X1X11X0	10101101	00101100	انطباق
0110XXXX	11110000	01100000	انطباق
1XXXXXXX	10000000	10000000	بدون انطباق
01101111	11111111	01101111	بدون انطباق

بررسی سطر اول به صورت دستی: $01101110 \oplus 01101000 = 00000110$ و $00000110 \wedge 01111000 = 00000000$ پس هیچ جایگاهی که قرار بوده مقایسه شود مخالفت نکرده است.

الگوهای صورت آزمایش ۸ بیتی‌اند و این TCAM ۱۶ بیتی است، پس بایت بالای هر الگو تماماً X گذاشته شده است. این یک دور زدن مسئله نیست؛ روش عادی ذخیره‌ی یک قاعده‌ی باریک در یک TCAM پهن‌تر همین است، و در تست‌بنچ با جست‌وجوی همان داده با دو بایت بالای متفاوت بررسی شده است.

چند انطباق هم‌زمان و اولویت

در مثال بالا یک کلمه‌ی جست‌وجو با سه قاعده منطبق شد. این حالت عادی است نه خطا — کل فایده‌ی TCAM همین است که قاعده‌های هم‌پوشان ذخیره شوند. اما چیزی باید انتخاب کند، و قرارداد تقریباً همه‌جایی این است که آدرس کوچک‌تر برنده است، تا قاعده‌های خاص‌تر جلوتر نوشته شوند.

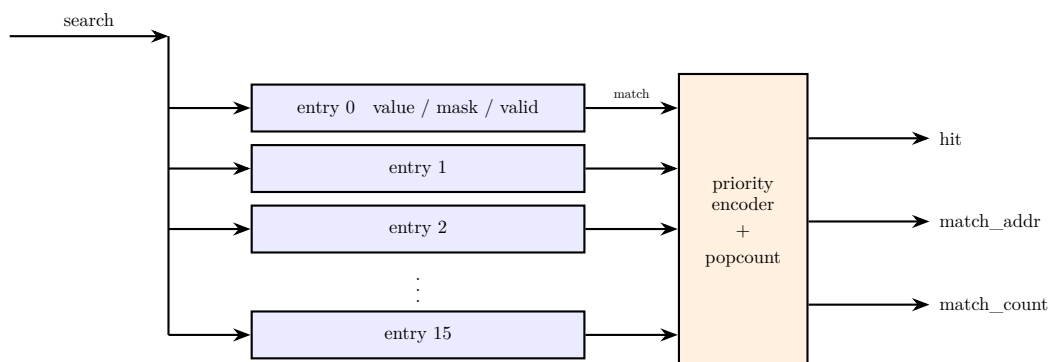
```

1 `include "tcam_pkg.vh"
2
3 // Lowest-index-wins priority encoder over the sixteen match lines.
4 //
5 // A TCAM can report several matches at once -- that is the normal case, not
6 // an error: 0110XXXX and 0X1X11X0 can both cover the same word. Something has
7 // to choose, and the near-universal convention is that the lower address
8 // wins, so more specific rules are simply stored earlier.
9 module priority_encoder (
10     input wire [TCAM_ENTRIES-1:0] lines,
11     output reg [TCAM_AW-1:0] index,
12     output wire any,
13 );
14     integer i;
15
16     assign any = |lines;
17
18     always @(*) begin
19         index = {TCAM_AW{1'b0}};
20         // Walking downwards means the last assignment taken is the lowest
21         // index that is set, so entry 0 has the highest priority.
22         for (i = TCAM_ENTRIES-1; i >= 0; i = i - 1)
23             if (lines[i]) index = i[TCAM_AW-1:0];
24     end
25 endmodule

```

علاوه بر آدرس برنده، تعداد خانه‌های منطبق هم بیرون داده می‌شود. این خروجی تزئینی نیست: تفاوت میان «یک قاعده این کلمه را پوشش می‌دهد» و «چهار قاعده آن را پوشش می‌دهند» اطلاعات واقعی است، و از نظر تست تنها راهی است که می‌شود کدگذار اولویت درست را از کدگذاری که اتفاقی خانه‌ی درست را انتخاب کرده تشخیص داد.

پیمانه‌ی کامل



کلمه‌ی جست‌وجو به همه‌ی خانه‌ها هم‌زمان پخش می‌شود و مسیر از ورودی تا خطوط انطباق کاملاً ترکیبی است. حافظه فقط در محتواست، نه در جست‌وجو؛ و همین است که یک CAM را CAM می‌کند: یک جست‌وجو به اندازه‌ی یک زنجیره‌ی تاخیر گیت طول می‌کشد، نه شانزده خواندن از حافظه.

در پیاده‌سازی روی FPGA معمولاً خروجی‌ها را ثابت‌دار می‌کنند تا بستن زمان‌بندی ساده شود. اینجا این کار عمداً انجام نشده تا پیمانه دقیقاً همان جست‌وجوی شرکت‌پذیری باشد که صورت آزمایش توصیف کرده، بدون هیچ چیز اضافه‌ای در مسیر.

```

1  `include "tcam_pkg.vh"
2
3  // A 16-entry, 16-bit-wide ternary CAM.
4  //
5  // Ports
6  // -----
7  // Write side: one entry at a time, given its value bits, its care mask and
8  // the validity bit. Writing wvalid = 0 retires an entry without having to
9  // clear its contents.
10 //
11 // Search side: purely combinational. `search` goes in, all sixteen entries
12 // are compared simultaneously, and three things come out:
13 //
14 //     match      the raw match lines, one per entry
15 //     hit / addr  the winning entry under lowest-index-wins priority
16 //     match_count HOW MANY entries matched
17 //
18 // match_count is not decoration. In a TCAM overlapping rules are expected, so
19 // the difference between "one rule covers this word" and "four rules do" is
20 // real information -- and it is the only way a testbench can tell a correct
21 // priority encoder from one that happens to pick the right entry by luck.
22 //
23 // On an FPGA the outputs would normally be registered to make timing closure
24 // easy. That is left out here so the module is exactly the associative lookup
25 // the lab sheet describes, with nothing else in the way.
26 module tcam (
27     input wire          Clk,
28     input wire          RstN,
29
30     input wire          we,
31     input wire [16:0]   waddr,
32     input wire [16:0]   wdata,
33     input wire [16:0]   wmask,
34     input wire          wvalid,
35
36     input wire [16:0]   search,
37     output wire [16:0]  match,
38     output wire         hit,
39     output wire [16:0]  match_addr,
40     output wire [16:0]  match_count,
41
42     // observation
43     input wire [16:0]   dbg_addr,
44     output wire [16:0]  dbg_value,
45     output wire [16:0]  dbg_mask,
46     output wire         dbg_valid
47 );
48 wire [16:0] q_value [0:15];
49 wire [16:0] q_mask [0:15];
50 wire [16:0] q_valid;
51
52 genvar e;

```

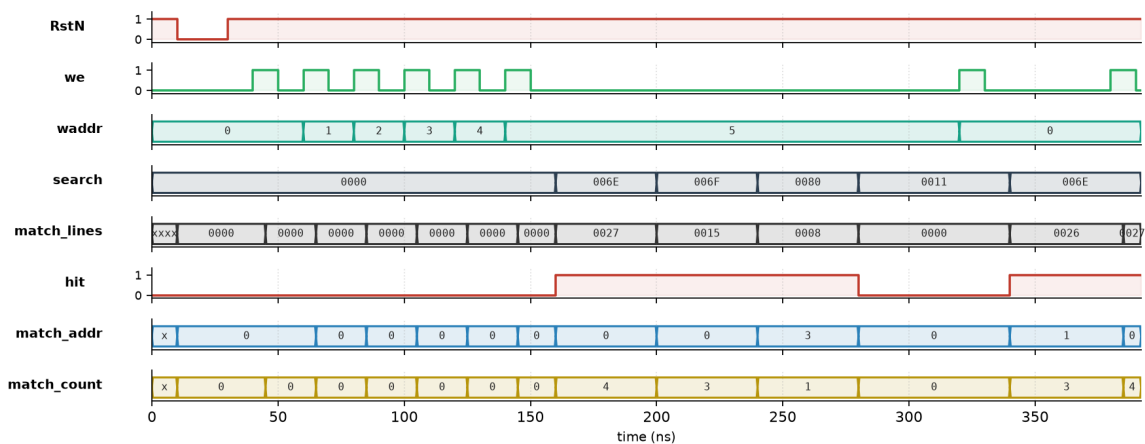
```

53 generate
54   for (e = 0; e < `TCAM_ENTRIES; e = e + 1) begin : g_entry
55     tcam_entry u_entry (
56       .Clk(Clk), .RstN(RstN),
57       .we(we && (waddr == e[`TCAM_AW-1:0])),
58       .wdata(wdata), .wmask(wmask), .wvalid(wvalid),
59       .search(search), .match(match[e]),
60       .q_value(q_value[e]), .q_mask(q_mask[e]), .q_valid(q_valid[e])
61     );
62   end
63 endgenerate
64
65 priority_encoder u_pri (
66   .lines(match), .index(match_addr), .any(hit)
67 );
68
69 // Population count of the match lines.
70 integer k;
71 reg [`TCAM_CNTW-1:0] cnt;
72 always @(*) begin
73   cnt = {'TCAM_CNTW{1'b0}};
74   for (k = 0; k < `TCAM_ENTRIES; k = k + 1)
75     cnt = cnt + match[k];
76 end
77 assign match_count = cnt;
78
79 assign dbg_value = q_value[dbg_addr];
80 assign dbg_mask = q_mask[dbg_addr];
81 assign dbg_valid = q_valid[dbg_addr];
82 endmodule

```

شکل موج

16 x 16 TCAM: one search word lights several match lines, lowest index wins



شش الگوی جدول بالا در خانه‌های ۰ تا ۵ نوشته شده‌اند و سپس چند کلمه جست‌وجو می‌شود:

جست‌وجو	match_lines	خانه‌های منطبق	count	addr
0x006E	0x0027	۰, ۱, ۲, ۵	۴	۰
0x006F	0x0015	۰, ۲, ۴	۳	۰
0x0080	0x0008	۳	۱	۳
0x0011	0x0000	—	۰	—
پس از بازنشسته کردن خانه‌ی ۰:				
0x006E	0x0026	۱, ۲, ۵	۳	۱

سطر اول همان مثال صورت آزمایش است. سطر دوم جالب‌تر است: 01101111 فقط در آخرین بیت با داده‌ی قبلی

فرق دارد، ولی مجموعه‌ی قاعده‌های منطبق کاملاً عوض می‌شود — 0X1X11X0 آن را رد می‌کند چون آخرین بیتش باید صفر باشد، و در عوض قاعده‌ی دقیق 01101111 حالا منطبق است.

دو سطر آخر رفتار اولویت را نشان می‌دهد: به محض بازنشسته شدن خانه‌ی ۰، خانه‌ی ۱ برنده می‌شود و شمارنده یکی کم می‌شود، بدون اینکه محتوای خانه‌ی ۰ پاک شده باشد.

توجه کنید که وقتی hit پایین است، مقدار match_addr بی‌معنی است. کدگذار اولویت در نبود هیچ انطباقی صفر بیرون می‌دهد و صفر یک آدرس معتبر است، پس مصرف‌کننده باید اول hit را ببیند. همین نکته باعث شد در تست‌بنچ، بررسی آدرس فقط زمانی انجام شود که مدل مرجع انتظار انطباق داشته باشد.

تست‌ها

چهار تست‌بنچ نوشته شده که از پایین به بالا مدار را می‌پوشانند.

(۱) سلول سه‌گانه — کامل. یک سلول فقط سه ورودی مؤثر دارد (مقدار ذخیره‌شده، بیت care و بیت جست‌وجو)، پس «کامل» اینجا واقعاً یعنی هر هشت ترکیب، در برابر جدول درستی‌ای که مستقیم از تعریف سه نماد $\{0, 1, X\}$ نوشته شده است.

```

1 `timescale 1ns/1ps
2
3 // The ternary cell, tested exhaustively.
4 //
5 // A single cell has only three inputs that matter -- the stored value, the
6 // stored care bit and the search bit -- so "exhaustive" here means all eight
7 // combinations, checked against the truth table written straight out of the
8 // definition of the three symbols {0, 1, X}.
9 module tb_tcam_cell;
10     reg Clk = 1'b0, RstN = 1'b1, we = 1'b0, d = 1'b0, care = 1'b0, s = 1'b0;
11     wire agree, d_q, care_q;
12
13     tcam_cell dut (
14         .Clk(Clk), .RstN(RstN), .we(we), .d(d), .care(care), .s(s),
15         .agree(agree), .d_q(d_q), .care_q(care_q)
16     );
17
18     always #5 Clk = ~Clk;
19
20     integer pass = 0, fail = 0;
21     integer vi, ci, si;
22     reg expect_agree;
23
24     task automatic chk;
25         input [511:0] name;
26         input ok;
27         begin
28             if (ok) pass = pass + 1;
29             else begin
30                 fail = fail + 1;
31                 $display(" FAIL @%t: %0s (d=%b care=%b s=%b agree=%b want=%b)",
32                     $time, name, d_q, care_q, s, agree, expect_agree);
33             end
34         end
35     endtask
36
37     task automatic store;
38         input v;
39         input c;
40         begin
41             @(negedge Clk); d = v; care = c; we = 1'b1;
42             @(negedge Clk); we = 1'b0;
43         end
44     endtask
45
46     initial begin
47         @(negedge Clk); RstN = 1'b0;
48         repeat (2) @(negedge Clk); RstN = 1'b1;
49         @(negedge Clk);
50
51         // After reset the cell holds X, which agrees with anything. This is
52         // exactly why the entry above it needs a validity bit.
53         chk("reset stores X (care = 0)", care_q === 1'b0);
54         s = 1'b0; #1; chk("reset cell agrees with 0", agree === 1'b1);
55         s = 1'b1; #1; chk("reset cell agrees with 1", agree === 1'b1);
56
57         // ---- all eight combinations ----
58         for (vi = 0; vi < 2; vi = vi + 1)

```

```

59   for (ci = 0; ci < 2; ci = ci + 1) begin
60       store(vi[0], ci[0]);
61       chk("stored value read back", d_q == vi[0]);
62       chk("stored care read back", care_q == ci[0]);
63       for (si = 0; si < 2; si = si + 1) begin
64           s = si[0]; #1;
65           // X agrees with everything; 0 and 1 agree only with themselves.
66           expect_agree = (ci[0] == 1'b0) ? 1'b1 : (vi[0] == si[0]);
67           chk("truth table", agree == expect_agree);
68       end
69   end
70
71   // ---- a cell is only rewritten when we is high ----
72   store(1'b1, 1'b1);
73   @(negedge Clk); d = 1'b0; care = 1'b1; we = 1'b0;
74   @(negedge Clk);
75   chk("no write without we", d_q == 1'b1 && care_q == 1'b1);
76
77   $display("tb_tcam_cell: Passed: %0d, Failed: %0d", pass, fail);
78   if (fail != 0) $fatal(1);
79   $finish;
80 end
81 endmodule

```

۲) یک خانه — جاروب کامل فضای جست‌وجو. یک خانه‌ی ۱۶ بیتی فقط ۶۵۵۳۶ کلمه‌ی جست‌وجوی ممکن دارد، پس برای هر الگوی ذخیره‌شده می‌شود همه‌ی آن‌ها را بررسی کرد؛ نه نمونه‌برداری و نه زیرمجموعه‌ی تصادفی. علاوه بر خود خط انطباق، تعداد کلمه‌های منطبق هم شمرده و با مقدار نظری مقایسه می‌شود. الگویی با n بیت X باید دقیقاً با 2^n کلمه منطبق شود:

الگو	بیت‌های X	تعداد کلمه‌های منطبق
mask = 0xFFFF	۰	۱
mask = 0xFF00	۸	۲۵۶
mask = 0x0F0F	۸	۲۵۶
mask = 0x8001	۱۴	۱۶۳۸۴
mask = 0x0000	۱۶	۶۵۵۳۶

این بررسی چیزی می‌گیرد که مقایسه‌ی تک‌تک نمی‌گیرد: اگر یک بیت care به سلول اشتباهی وصل شده باشد، برای بیشتر کلمه‌ها پاسخ همچنان درست درمی‌آید ولی اندازه مجموعه‌ی منطبق عوض می‌شود.

```

1  `timescale 1ns/1ps
2  `include "tcam_pkg.vh"
3
4  // One 16-bit entry, swept over EVERY possible search word.
5  //
6  // A 16-bit entry has only 65536 possible search words, so for a given stored
7  // pattern the match line can be checked against the model for all of them --
8  // no sampling, no random subset. Several patterns are swept this way,
9  // including the two extremes: a fully specified pattern (an ordinary CAM
10 // entry, exactly one word matches) and an all-X pattern (every word matches).
11 module tb_tcam_entry;
12     reg                Clk = 1'b0, RstN = 1'b1;
13     reg                we = 1'b0, wvalid = 1'b1;
14     reg [ `TCAM_WIDTH-1:0 ] wdata = 16'd0, wmask = 16'd0;
15     reg [ `TCAM_WIDTH-1:0 ] search = 16'd0;
16
17     wire                match;
18     wire [ `TCAM_WIDTH-1:0 ] q_value, q_mask;
19     wire                q_valid;
20
21     tcam_entry dut (
22         .Clk(Clk), .RstN(RstN),
23         .we(we), .wdata(wdata), .wmask(wmask), .wvalid(wvalid),
24         .search(search), .match(match),
25         .q_value(q_value), .q_mask(q_mask), .q_valid(q_valid)
26     );
27
28     always #5 Clk = ~Clk;
29
30     integer pass = 0, fail = 0;
31     integer p, w, hits;
32     reg        expect_match;

```

```

33
34 task automatic chk;
35     input [511:0] name;
36     input      ok;
37     begin
38         if (ok) pass = pass + 1;
39         else begin
40             fail = fail + 1;
41             $display(" FAIL @%0t: %0s (value=%h mask=%h search=%h match=%b want=%b)",
42                 $time, name, q_value, q_mask, search, match, expect_match);
43         end
44     end
45 endtask
46
47 task automatic store;
48     input [`TCAM_WIDTH-1:0] v;
49     input [`TCAM_WIDTH-1:0] m;
50     input      vld;
51     begin
52         @(negedge Clk); wdata = v; wmask = m; wvalid = vld; we = 1'b1;
53         @(negedge Clk); we = 1'b0;
54         #1;
55     end
56 endtask
57
58 // Sweep every 16-bit search word against one stored pattern.
59 task automatic sweep;
60     input [`TCAM_WIDTH-1:0] v;
61     input [`TCAM_WIDTH-1:0] m;
62     input integer      want_hits;    // -1 = do not check the total
63     begin
64         store(v, m, 1'b1);
65         hits = 0;
66         for (w = 0; w < 65536; w = w + 1) begin
67             search = w[`TCAM_WIDTH-1:0];
68             #1;
69             expect_match = (((w[`TCAM_WIDTH-1:0] ^ v) & m) == `{TCAM_WIDTH{1'b0}});
70             if (match != expect_match) begin
71                 fail = fail + 1;
72                 $display(" FAIL @%0t: sweep value=%h mask=%h search=%h match=%b want=%b",
73                     $time, v, m, search, match, expect_match);
74             end else pass = pass + 1;
75             if (match) hits = hits + 1;
76         end
77         if (want_hits >= 0) begin
78             expect_match = 1'bx;
79             chk("number of matching words over the whole space",
80                 hits == want_hits);
81         end
82     end
83 endtask
84
85 initial begin
86     @(negedge Clk); RstN = 1'b0;
87     repeat (2) @(negedge Clk); RstN = 1'b1;
88     @(negedge Clk); #1;
89
90     // An entry that was never written must never match, even though its
91     // cells all hold X.
92     chk("entry invalid after reset", q_valid == 1'b0);
93     search = 16'hA5A5; #1; chk("invalid entry does not match", match == 1'b0);
94     search = 16'h0000; #1; chk("invalid entry does not match zero", match == 1'b0);
95
96     // ---- fully specified: behaves like an ordinary CAM entry ----
97     sweep(16'h1234, 16'hFFFF, 1);
98
99     // ---- all X: matches the entire space; only `valid` holds it back ----
100    sweep(16'h0000, 16'h0000, 65536);
101
102    // ---- eight care bits: 2^8 = 256 words match ----
103    sweep(16'hAB00, 16'hFF00, 256);
104
105    // ---- scattered care bits ----
106    sweep(16'h5A5A, 16'h0F0F, 256);
107    sweep(16'hFFFF, 16'h8001, 16384);
108
109    // ---- retiring an entry ----
110    store(16'h1234, 16'hFFFF, 1'b1);
111    search = 16'h1234; #1;
112    chk("stored word matches", match == 1'b1);
113    store(16'h1234, 16'hFFFF, 1'b0);
114    search = 16'h1234; #1;
115    chk("retired entry stops matching", match == 1'b0);
116    chk("contents survive retirement", q_value == 16'h1234 && q_mask == 16'hFFFF);
117
118    $display("tb_tcam_entry: Passed: %0d, Failed: %0d", pass, fail);
119    if (fail != 0) $fatal(1);

```



```

120     $finish;
121 end
122 endmodule

```

۳) پیمانه‌ی کامل در برابر مدل مرجع. مدل چند خط رفتاری بدون سلول، بدون کدگذار اولویت و بدون درخت شمارش است، پس نمی‌تواند اشتباه ساختاری طراحی را تکرار کند.

مهم‌تر از خود مدل، نحوه‌ی راندن آن است: هم محتوای TCAM و هم کلمه‌های جست‌وجو تصادفی‌اند، و ماسک‌ها عمداً به سمت داشتن X زیاد سوگیری شده‌اند (\$random & \$random). دلیلش این است که رفتار جالب TCAM وقتی ظاهر می‌شود که قاعده‌ها هم‌پوشانی داشته باشند، و مجموعه‌ی بردارهای دست‌نویس دقیقاً از هم‌پوشانی پرهیز می‌کند. در پایان، یک پیکربندی هم روی تمام ۶۵۵۳۶ کلمه‌ی ممکن جاروب می‌شود تا دست‌کم یک نقشه‌ی کامل از فضای انطباق بررسی شده باشد، نه نمونه‌برداری شده.

```

1  `timescale 1ns/1ps
2  `include "tcam_pkg.vh"
3
4  // The whole 16 x 16 TCAM against a reference model.
5  //
6  // The model is three lines of behavioural code with no cells, no priority
7  // encoder and no popcount tree, so it cannot repeat a structural mistake the
8  // design makes. It is driven with random contents AND random searches,
9  // because the interesting behaviour of a TCAM is what happens when stored
10 // patterns OVERLAP -- and overlap is exactly what a hand-written set of test
11 // vectors tends to avoid.
12 //
13 // One configuration is also swept over all 65536 possible search words, so at
14 // least one complete map of the match space is verified rather than sampled.
15 module tb_tcam;
16     reg                Clk = 1'b0, RstN = 1'b1;
17     reg                we = 1'b0, wvalid = 1'b1;
18     reg [TCAM_AW-1:0] waddr = 4'd0;
19     reg [TCAM_WIDTH-1:0] wdata = 16'd0, wmask = 16'd0;
20     reg [TCAM_WIDTH-1:0] search = 16'd0;
21     reg [TCAM_AW-1:0] dbg_addr = 4'd0;
22
23     wire [TCAM_ENTRIES-1:0] match;
24     wire                    hit;
25     wire [TCAM_AW-1:0] match_addr;
26     wire [TCAM_CNTW-1:0] match_count;
27     wire [TCAM_WIDTH-1:0] dbg_value, dbg_mask;
28     wire                    dbg_valid;
29
30     tcam dut (
31         .Clk(Clk), .RstN(RstN),
32         .we(we), .waddr(waddr), .wdata(wdata), .wmask(wmask), .wvalid(wvalid),
33         .search(search), .match(match), .hit(hit),
34         .match_addr(match_addr), .match_count(match_count),
35         .dbg_addr(dbg_addr), .dbg_value(dbg_value), .dbg_mask(dbg_mask),
36         .dbg_valid(dbg_valid)
37     );
38
39     always #5 Clk = ~Clk;
40
41     integer pass = 0, fail = 0;
42     integer c, e, w, seed;
43
44     // model copy of the contents
45     reg [TCAM_WIDTH-1:0] m_val [0:TCAM_ENTRIES-1];
46     reg [TCAM_WIDTH-1:0] m_mask [0:TCAM_ENTRIES-1];
47     reg [TCAM_ENTRIES-1:0] m_vld;
48
49     reg [TCAM_ENTRIES-1:0] exp_match;
50     reg [TCAM_AW-1:0] exp_addr;
51     reg [TCAM_CNTW-1:0] exp_count;
52     reg exp_hit;
53
54     task automatic chk;
55         input [511:0] name;
56         input ok;
57         begin
58             if (ok) pass = pass + 1;
59             else begin
60                 fail = fail + 1;
61                 $display(" FAIL @%0t: %0s (search=%h match=%h/%h hit=%b/%b addr=%0d/%0d cnt=%0d/%0d)",
62                     $time, name, search, match, exp_match, hit, exp_hit,
63                     match_addr, exp_addr, match_count, exp_count);
64             end
65         end
66     endtask
67

```

```

68 task automatic store;
69   input [`TCAM_AW-1:0] a;
70   input [`TCAM_WIDTH-1:0] v;
71   input [`TCAM_WIDTH-1:0] m;
72   input vld;
73   begin
74     @(negedge Clk);
75     waddr = a; wdata = v; wmask = m; wvalid = vld; we = 1'b1;
76     @(negedge Clk); we = 1'b0;
77     m_val[a] = v;
78     m_mask[a] = m;
79     m_vld[a] = vld;
80     #1;
81   end
82 endtask
83
84 // Recompute what the TCAM should report for the current `search`.
85 task automatic model;
86   integer g;
87   begin
88     exp_match = {`TCAM_ENTRIES{1'b0}};
89     exp_count = {`TCAM_CNTW{1'b0}};
90     for (g = 0; g < `TCAM_ENTRIES; g = g + 1)
91       if (m_vld[g] && ((search ^ m_val[g]) & m_mask[g]) == {`TCAM_WIDTH{1'b0}})) begin
92         exp_match[g] = 1'b1;
93         exp_count = exp_count + 1'b1;
94       end
95     exp_hit = |exp_match;
96     exp_addr = {`TCAM_AW{1'b0}};
97     for (g = `TCAM_ENTRIES-1; g >= 0; g = g - 1)
98       if (exp_match[g]) exp_addr = g[`TCAM_AW-1:0];
99   end
100 endtask
101
102 task automatic probe;
103   input [`TCAM_WIDTH-1:0] s;
104   begin
105     search = s; #1;
106     model;
107     chk("match lines", match == exp_match);
108     chk("hit", hit == exp_hit);
109     chk("count", match_count == exp_count);
110     // The winning address is only defined when something matched.
111     if (exp_hit) chk("priority address", match_addr == exp_addr);
112   end
113 endtask
114
115 initial begin
116   @(negedge Clk); RstN = 1'b0;
117   repeat (2) @(negedge Clk); RstN = 1'b1;
118   @(negedge Clk);
119
120   for (e = 0; e < `TCAM_ENTRIES; e = e + 1) begin
121     m_val[e] = 16'd0;
122     m_mask[e] = 16'd0;
123     m_vld[e] = 1'b0;
124   end
125   #1;
126
127   // ---- an empty TCAM must be silent, despite every cell holding X ----
128   probe(16'h0000);
129   probe(16'hFFFF);
130   probe(16'h5A5A);
131   chk("empty TCAM never hits", hit == 1'b0 && match_count == 5'd0);
132
133   // ---- write-back check on the stored contents ----
134   store(4'd7, 16'h1234, 16'hFF0F, 1'b1);
135   dbg_addr = 4'd7; #1;
136   chk("value stored", dbg_value == 16'h1234);
137   chk("mask stored", dbg_mask == 16'hFF0F);
138   chk("valid stored", dbg_valid == 1'b1);
139   dbg_addr = 4'd6; #1;
140   chk("neighbouring entry untouched", dbg_valid == 1'b0);
141
142   // ---- random contents, random searches ----
143   seed = 32'd90909;
144   for (c = 0; c < 60; c = c + 1) begin
145     for (e = 0; e < `TCAM_ENTRIES; e = e + 1) begin
146       // Masks are biased towards having many X positions, which is
147       // what makes entries overlap and the priority encoder matter.
148       store(e[`TCAM_AW-1:0],
149         $random(seed),
150         {$random(seed)} & {$random(seed)},
151         (($random(seed)) % 8) != 0));
152     end
153     for (w = 0; w < 200; w = w + 1)
154       probe($random(seed));

```

```

155     end
156
157     // ---- one configuration swept over the entire search space ----
158     for (e = 0; e < `TCAM_ENTRIES; e = e + 1)
159         store(e[`TCAM_AW-1:0],
160             $random(seed),
161             {$random(seed)} & {$random(seed)},
162             1'b1);
163     for (w = 0; w < 65536; w = w + 1)
164         probe(w[`TCAM_WIDTH-1:0]);
165
166     $display("tb_tcam: Passed: %0d, Failed: %0d", pass, fail);
167     if (fail != 0) $fatal(1);
168     $finish;
169 end
170 endmodule

```

۴) انطباق مستقیم با صورت آزمایش. در این تست‌بنچ الگوها به صورت متن و با همان نماد $\{0, 1, X\}$ صورت آزمایش نوشته می‌شوند و خود تست‌بنچ آن‌ها را به زوج مقدار/ماسک تبدیل می‌کند:

```

store_pattern(4'd0, "XXXXXXXX1101XXX");
store_pattern(4'd1, "XXXXXXXX0X11X0");
store_pattern(4'd2, "XXXXXXXX0110XXXX");

```

این کار دو فایده دارد. اول اینکه بررسی‌ها کنار صورت آزمایش خوانا می‌مانند. دوم و مهم‌تر اینکه خود رمزگذاری مقدار/ماسک بخشی از چیزی می‌شود که آزموده می‌شود: اگر رمزگذاری غلط بود، مثال کار شده‌ی صورت آزمایش از انطباق می‌افتاد.

```

1 `timescale 1ns/1ps
2 `include "tcam_pkg.vh"
3
4 // Direct checks against the lab sheet, with no reference model involved.
5 //
6 // Patterns are written here as TEXT, in the same {0, 1, X} notation the sheet
7 // uses, and converted to a value/mask pair by the testbench. That keeps the
8 // checks readable next to the sheet and makes the value/mask encoding itself
9 // part of what is being verified: if the encoding were wrong, the worked
10 // example from the sheet would stop matching.
11 //
12 // The sheet's example is 8 bits wide and this TCAM is 16, so each pattern is
13 // written with its high byte marked entirely as X. That is not a workaround --
14 // it is the ordinary way a narrow rule is stored in a wider TCAM, and it gets
15 // checked here by searching with two different high bytes.
16 module tb_tcam_spec;
17     reg Clk = 1'b0, RstN = 1'b1;
18     reg we = 1'b0, wvalid = 1'b1;
19     reg [`TCAM_AW-1:0] waddr = 4'd0;
20     reg [`TCAM_WIDTH-1:0] wdata = 16'd0, wmask = 16'd0;
21     reg [`TCAM_WIDTH-1:0] search = 16'd0;
22     reg [`TCAM_AW-1:0] dbg_addr = 4'd0;
23
24     wire [`TCAM_ENTRIES-1:0] match;
25     wire hit;
26     wire [`TCAM_AW-1:0] match_addr;
27     wire [`TCAM_CNTW-1:0] match_count;
28     wire [`TCAM_WIDTH-1:0] dbg_value, dbg_mask;
29     wire dbg_valid;
30
31     tcam dut (
32         .Clk(Clk), .RstN(RstN),
33         .we(we), .waddr(waddr), .wdata(wdata), .wmask(wmask), .wvalid(wvalid),
34         .search(search), .match(match), .hit(hit),
35         .match_addr(match_addr), .match_count(match_count),
36         .dbg_addr(dbg_addr), .dbg_value(dbg_value), .dbg_mask(dbg_mask),
37         .dbg_valid(dbg_valid)
38     );
39
40     always #5 Clk = ~Clk;
41
42     integer pass = 0, fail = 0;
43
44     task automatic chk;
45         input [511:0] name;
46         input ok;
47         begin
48             if (ok) pass = pass + 1;
49             else begin
50                 fail = fail + 1;
51                 $display(" FAIL @%0t: %0s (search=%h match=%h hit=%b addr=%0d cnt=%0d)",
52                     $time, name, search, match, hit, match_addr, match_count);
53             end
54         end
55     endtask

```

```

54     end
55 endtask
56
57 task automatic store;
58     input [`TCAM_AW-1:0] a;
59     input [`TCAM_WIDTH-1:0] v;
60     input [`TCAM_WIDTH-1:0] m;
61     input vld;
62     begin
63         @(negedge Clk);
64         waddr = a; wdata = v; wmask = m; wvalid = vld; we = 1'b1;
65         @(negedge Clk); we = 1'b0;
66         #1;
67     end
68 endtask
69
70 // Store a pattern written as 16 characters of '0', '1' or 'X'.
71 // The leftmost character is the most significant bit.
72 task automatic store_pattern;
73     input [`TCAM_AW-1:0] a;
74     input [8*`TCAM_WIDTH-1:0] pat;
75     reg [`TCAM_WIDTH-1:0] v, m;
76     reg [7:0] ch;
77     integer i;
78     begin
79         v = {`TCAM_WIDTH{1'b0}};
80         m = {`TCAM_WIDTH{1'b0}};
81         for (i = 0; i < `TCAM_WIDTH; i = i + 1) begin
82             ch = pat[8*(`TCAM_WIDTH-1-i) +: 8]; // i-th character
83             if (ch == "1") begin
84                 v[`TCAM_WIDTH-1-i] = 1'b1;
85                 m[`TCAM_WIDTH-1-i] = 1'b1;
86             end else if (ch == "0") begin
87                 m[`TCAM_WIDTH-1-i] = 1'b1; // value stays 0
88             end
89             // anything else is X: care stays 0
90         end
91         store(a, v, m, 1'b1);
92     end
93 endtask
94
95 task automatic look;
96     input [`TCAM_WIDTH-1:0] s;
97     begin
98         search = s; #1;
99     end
100 endtask
101
102 initial begin
103     @(negedge Clk); RstN = 1'b0;
104     repeat (2) @(negedge Clk); RstN = 1'b1;
105     @(negedge Clk); #1;
106
107     // =====
108     // The worked example from the lab sheet.
109     //
110     // searching for 01101110, the sheet says these three are matches:
111     // X1101XXX 0X1X11X0 0110XXXX
112     // =====
113     store_pattern(4'd0, "XXXXXXXXX1101XXX"); // sheet: match
114     store_pattern(4'd1, "XXXXXXXXX0X1X11X0"); // sheet: match
115     store_pattern(4'd2, "XXXXXXXXX0110XXXX"); // sheet: match
116     store_pattern(4'd3, "XXXXXXXXX1XXXXXXX"); // MSB is wrong: no match
117     store_pattern(4'd4, "XXXXXXXXX01101111"); // differs in the last bit
118     store_pattern(4'd5, "XXXXXXXXX01101110"); // exact copy of the data
119
120     // check the value/mask encoding of the first pattern by hand:
121     // X 1 1 0 1 X X X -> care = 0111000, value = 01101000
122     dbg_addr = 4'd0; #1;
123     chk("pattern X1101XXX encodes to mask 0x0078", dbg_mask === 16'h0078);
124     chk("pattern X1101XXX encodes to value 0x0068", dbg_value === 16'h0068);
125
126     look(16'h006E); // the data word 01101110
127     chk("sheet: X1101XXX matches", match[0] === 1'b1);
128     chk("sheet: 0X1X11X0 matches", match[1] === 1'b1);
129     chk("sheet: 0110XXXX matches", match[2] === 1'b1);
130     chk("1XXXXXX does not match", match[3] === 1'b0);
131     chk("01101111 does not match", match[4] === 1'b0);
132     chk("01101110 matches exactly", match[5] === 1'b1);
133     chk("four entries matched in total", match_count === 5'd4);
134     chk("there was a hit", hit === 1'b1);
135     chk("lowest matching index wins", match_addr === 4'd0);
136
137     // The high byte of every pattern is X, so it really is ignored.
138     look(16'hFF6E);
139     chk("high byte is don't-care: same match lines",
140         match[5:0] === 6'b100111 && match_count === 5'd4);

```

```

141 look(16'h5A6E);
142 chk("high byte is don't-care: any value works", match_count == 5'd4);
143
144 // =====
145 // A neighbouring data word: 01101111 differs only in the last bit.
146 // =====
147 look(16'h006F);
148 chk("X1101XXX still matches 01101111", match[0] == 1'b1);
149 chk("0X1X11X0 rejects it (last bit must be 0)", match[1] == 1'b0);
150 chk("0110XXXX still matches", match[2] == 1'b1);
151 chk("01101111 now matches exactly", match[4] == 1'b1);
152 chk("01101110 no longer matches", match[5] == 1'b0);
153 chk("three entries matched", match_count == 5'd3);
154
155 // =====
156 // Priority is lowest index wins, and retiring the winner promotes
157 // the next one.
158 // =====
159 look(16'h006E);
160 chk("winner is entry 0", match_addr == 4'd0);
161 store(4'd0, 16'h0068, 16'h0078, 1'b0); // retire entry 0
162 look(16'h006E);
163 chk("after retiring entry 0 the winner is entry 1", match_addr == 4'd1);
164 chk("and one fewer entry matches", match_count == 5'd3);
165 store(4'd0, 16'h0068, 16'h0078, 1'b1); // bring it back
166 look(16'h006E);
167 chk("re-validating restores the winner", match_addr == 4'd0);
168 chk("and the match count", match_count == 5'd4);
169
170 // =====
171 // The two extremes of a ternary entry.
172 // =====
173 store_pattern(4'd15, "XXXXXXXXXXXXXX"); // all X
174 look(16'h0000); chk("all-X entry matches 0x0000", match[15] == 1'b1);
175 look(16'hFFFF); chk("all-X entry matches 0xFFFF", match[15] == 1'b1);
176 look(16'hA5A5); chk("all-X entry matches anything", match[15] == 1'b1);
177
178 store_pattern(4'd14, "0000111100001111"); // fully specified
179 look(16'h0F0F); chk("exact entry matches its word", match[14] == 1'b1);
180 look(16'h0F0E); chk("exact entry rejects a neighbour", match[14] == 1'b0);
181
182 // The validity bit is what keeps an all-X entry from swallowing every
183 // search: without it, an unprogrammed TCAM would match everything.
184 store(4'd15, 16'h0000, 16'h0000, 1'b0);
185 look(16'hA5A5);
186 chk("retired all-X entry is silent", match[15] == 1'b0);
187
188 // =====
189 // Nothing at all matches -> no hit.
190 // =====
191 @(negedge Clk); RstN = 1'b0;
192 repeat (2) @(negedge Clk); RstN = 1'b1;
193 @(negedge Clk);
194 store_pattern(4'd0, "11111111111111");
195 look(16'h0000);
196 chk("no match means no hit", hit == 1'b0 && match_count == 5'd0);
197 chk("match lines are all clear", match == 16'h0000);
198
199 $display("tb_tcam_spec: Passed: %0d, Failed: %0d", pass, fail);
200 if (fail != 0) $fatal(1);
201 $finish;
202
203 end
endmodule

```

نتایج تست

```

iverilog: Icarus Verilog version 12.0 (stable) ()

=== tb_tcam_cell ===
tb_tcam_cell: Passed: 20, Failed: 0
PASS: tb_tcam_cell

=== tb_tcam_entry ===
tb_tcam_entry: Passed: 327691, Failed: 0
PASS: tb_tcam_entry

=== tb_tcam ===
tb_tcam: Passed: 300532, Failed: 0
PASS: tb_tcam

=== tb_tcam_spec ===
tb_tcam_spec: Passed: 32, Failed: 0
PASS: tb_tcam_spec

```

summary: 4/4 passed, 0 failed

جمع‌بندی

TCAM از نظر منطقی ساده است — دو فلیپ‌فلاپ و سه گیت به ازای هر بیت — ولی دو نکته‌ی طراحی دارد که هیچ‌کدام از خود فرمول مقایسه بیرون نمی‌آید.

اول، بیت اعتبار. چون نماد X با نبود care کدگذاری می‌شود، حالت خالی مدار به‌طور طبیعی «با همه‌چیز منطبق شو» است نه «با هیچ‌چیز منطبق نشو». یعنی امن‌ترین حالت پس از ریست، بدترین حالت رفتاری است، و بیت اعتبار تنها چیزی است که آن را برمی‌گرداند.

دوم، اولویت. برخلاف حافظه‌ی معمولی که هر آدرس یک پاسخ دارد، پاسخ TCAM می‌تواند چندتایی باشد. این ابهام ذاتی مسئله است، نه اشکال پیاده‌سازی، و باید با یک قرارداد صریح — اینجا «آدرس کوچک‌تر برنده است» — حل شود. خروجی `match_count` برای همین اضافه شد: بدون آن، تست‌بنچ نمی‌توانست تشخیص دهد که کدگذار اولویت درست کار می‌کند یا فقط اتفاقی خانه‌ی درست را برگردانده است.